Done

Musical Track Database plus Artists

This application will read an iTunes library in comma-separated-values (CSV) and produce properly normalized tables as specified below. Once you have placed the proper data in the tables, press the button below to check your answer.

Host: pg.pg4e.com

Port: 5432

Database: pg4e_3774c65c52

User: pg4e_3774c65c52

Password: (hide/show copy)

Check Answer

Here is the the **psql** command to access your database:

```
psql -h pg.pg4e.com -p 5432 -U pg4e_3774c65c52 pg4e_3774c65c52
```

We will do some things differently in this assignment. We will not use a separate "raw" table, we will just use **ALTER TABLE** statements to remove columns after we don't need them (i.e. we converted them into foreign keys).

We will use the same CSV track data (library.csv?PHPSESSID=79f98c36cf83e0692541c619e79fac88") as in prior exercises - this time we will build a many-to-many relationship using a *junction/through/join* table between tracks and artists.

To grade this assignment, the program will run a query like this on your database and look for the data it expects to see:

```
SELECT track.title, album.title, artist.name
FROM track
JOIN album ON track.album_id = album.id
JOIN tracktoartist ON track.id = tracktoartist.track_id
JOIN artist ON tracktoartist.artist_id = artist.id
ORDER BY track.title
LIMIT 3;
```

The expected result of this query on your database is:

title	album	artist
A Boy Named Sue (live)	The Legend Of Johnny Cash	Johnny Cash
A Brief History of Packets	Computing Conversations	IEEE Computer Society

Aguas De Marco	Natural Wonders Music Sampler 1999	Rosa Passos

Done

In this assignment we will give you a partial script with portions of some of the commands replaced by three dots...

Done

```
DROP TABLE album CASCADE;
CREATE TABLE album (
    id SERIAL,
    title VARCHAR(128) UNIQUE,
    PRIMARY KEY(id)
);

DROP TABLE track CASCADE;
CREATE TABLE track (
    id SERIAL,
    title TEXT,
    artist TEXT,
    album TEXT,
    album_id INTEGER REFERENCES album(id) ON DELETE CASCADE,
    count INTEGER,
    rating INTEGER,
    len INTEGER,
    PRIMARY KEY(id)
);

DROP TABLE artist CASCADE;
CREATE TABLE artist (
    id SERIAL,
    name VARCHAR(128) UNIQUE,
    PRIMARY KEY(id)
);

DROP TABLE tracktoartist CASCADE;
CREATE TABLE tracktoartist (
    id SERIAL,
    track VARCHAR(128),
    track_id INTEGER REFERENCES track(id) ON DELETE CASCADE,
    artist VARCHAR(128),
    artist_id INTEGER REFERENCES artist(id) ON DELETE CASCADE,
    PRIMARY KEY(id)
);

\copy track(title,artist,album,count,rating,len) FROM 'library.csv' WITH DELIMITER ',' CSV;

INSERT INTO album (title) SELECT DISTINCT album FROM track;
UPDATE track SET album_id = (SELECT album.id FROM album WHERE album.title = track.album);

INSERT INTO tracktoartist (track, artist) SELECT DISTINCT ...

INSERT INTO artist (name) ...

UPDATE tracktoartist SET track_id = ...
UPDATE tracktoartist SET artist_id = ...

-- We are now done with these text fields
ALTER TABLE track DROP COLUMN album;
ALTER TABLE track ...
```

```
ALTER TABLE tracktoartist DROP COLUMN track;
ALTER TABLE tracktoartist ...

SELECT track.title, album.title, artist.name
FROM track
JOIN album ON track.album_id = album.id
JOIN tracktoartist ON track.id = tracktoartist.track_id
JOIN artist ON tracktoartist.artist_id = artist.id
LIMIT 3;
```

Done